

Transistors as switches

[Circuit 1](#)

[Circuit 2](#)



Embedded Programming

Computer architecture

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

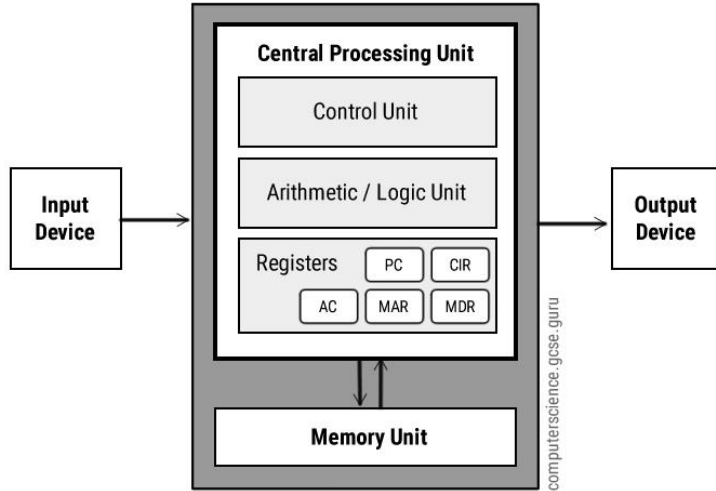
Maastricht University

2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl



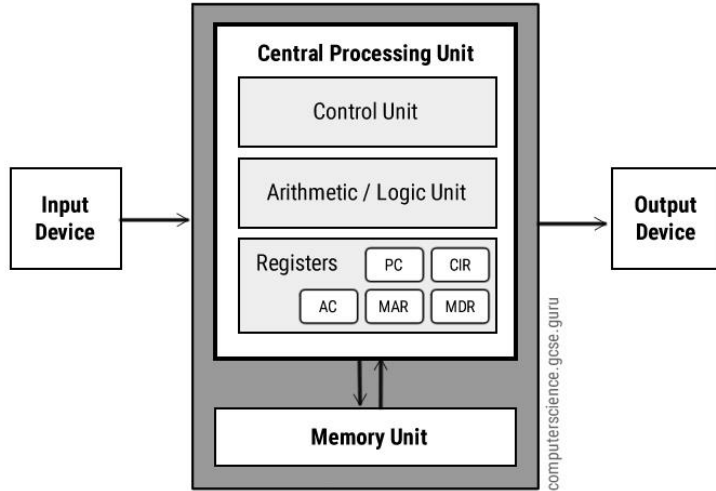
Computer Architecture



<https://www.computerscience.gcse.guru/>

- The Von Neumann architecture (also known as the von Neumann model or Princeton architecture) was first published by John von Neumann in 1945
- His computer architecture design consists of a Control Unit, Arithmetic and Logic Unit (ALU), Memory Unit, Registers and Inputs/Outputs
- The Von Neumann architecture is based on the *stored-program* computer concept, where instruction data and program data are stored in the same memory. This design is still used in most computers produced today

Computer Architecture



<https://www.computerscience.gcse.guru/>

The term "von Neumann architecture" has evolved to refer to **any** stored-program computer in which:

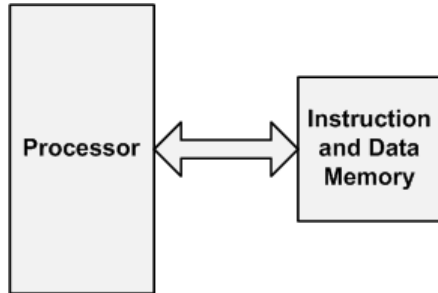
- an instruction fetch
- and a data operation

cannot occur at the same time since they share a common bus

Computer Architecture

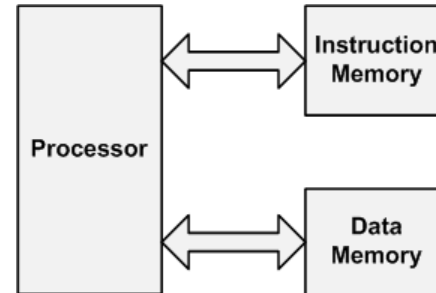
Von-Neumann

Instructions and data are stored in the same memory



Harvard

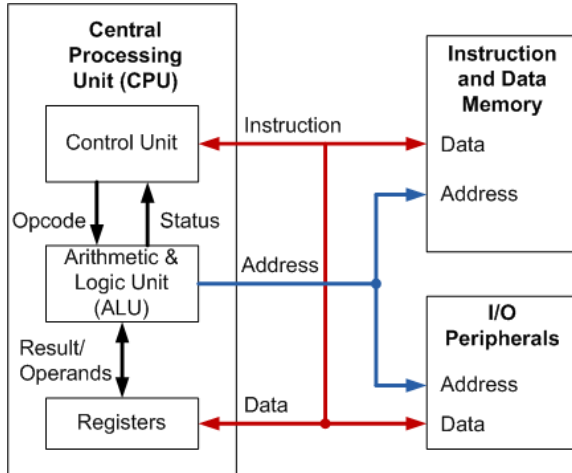
Data and instructions are stored into separate memories



Computer Architecture

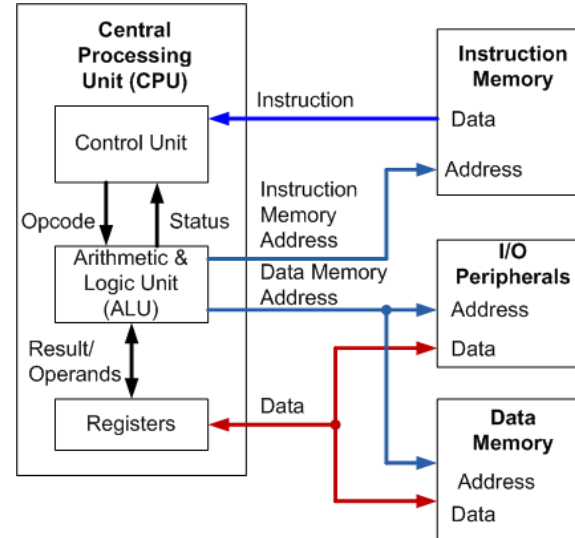
Von-Neumann

Instructions and data are stored in the same memory



Harvard

Data and instructions are stored into separate memories



Processor and datapaths

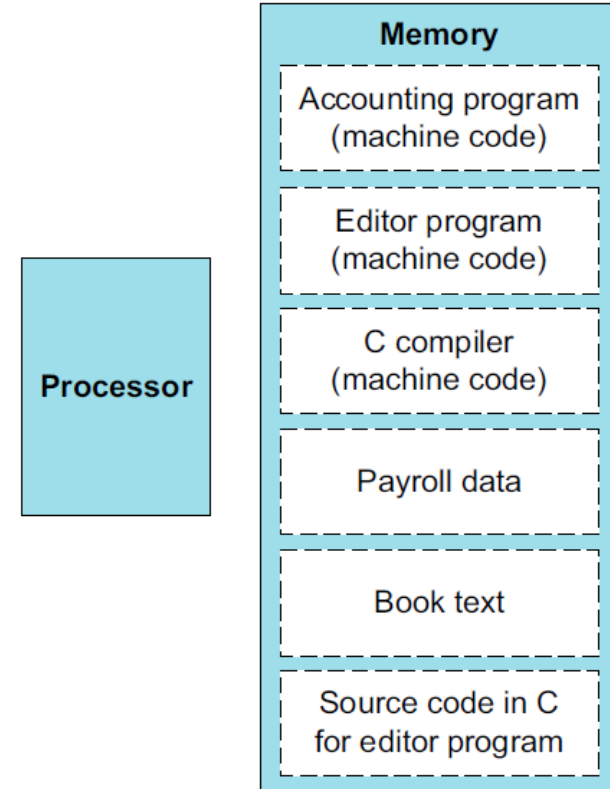
The BIG Picture

Today's computers are built on two key principles:

1. Instructions are represented as numbers
2. Programs are stored in memory to be read or written, just like data

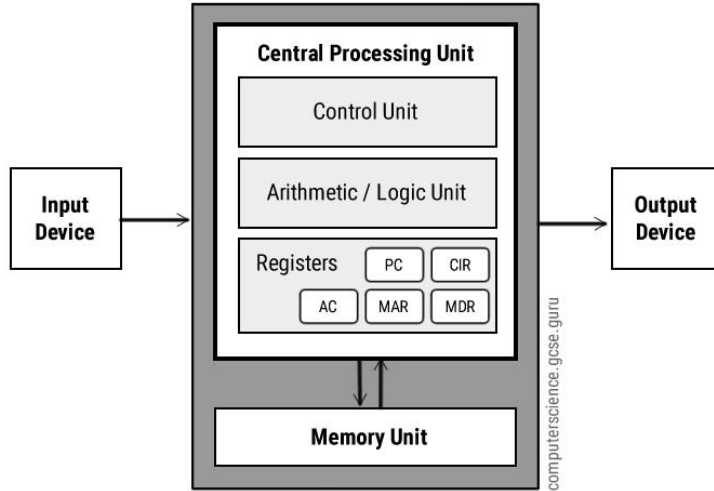
These principles lead to the *stored-program* concept:

- One consequence of instructions as numbers is that programs are often shipped as files of binary numbers
- The commercial implication is that computers can inherit ready-made software provided they are compatible with an existing instruction set



Computer Architecture

Let's zoom in on some of the main components



<https://www.computerscience.gcse.guru/>

Control flow

- Processors do only one thing:
 - From startup to shutdown, a CPU simply reads and executes (interprets) a sequence of instructions, one at a time
- This sequence is the CPU's **control flow**

Physical control flow

Time



<startup>

inst₁

inst₂

inst₃

...

inst_n

<shutdown>

Control flow

The key stages in a processor's control flow include:

- **Instruction Fetch (IF):** Retrieve the next instruction from memory, (typically from the PC), which keeps track of the instruction sequence
- **Instruction Decode (ID):** Interpret the fetched instruction, determining what needs to be done. The instruction is broken down into opcode (operation code) and operands (data or memory locations)
- **Execution (EX):** The ALU performs mathematical operations or logic functions, depending on the decoded instruction. If the instruction involves memory access, the processor determines the address.

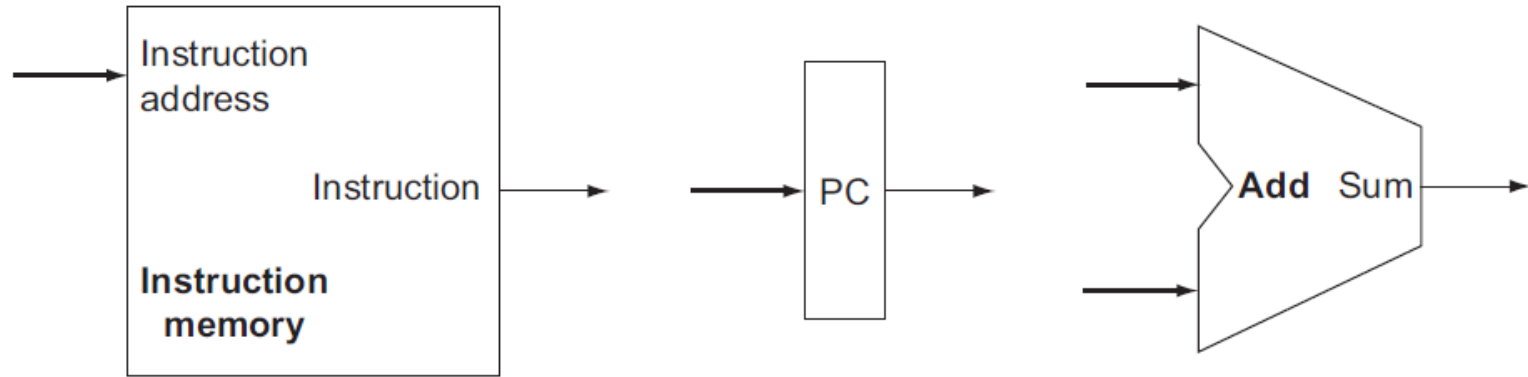


Program counter - PC

- Implicit in the *stored-program* idea is the need to have a register to hold the address of the current instruction being executed
- For historical reasons, this register is almost always called the **program counter**, abbreviated **PC** (although a more sensible name would have been instruction address register)

program counter (PC) The register containing the address of the instruction in the program being executed.

Main components for simple instructions

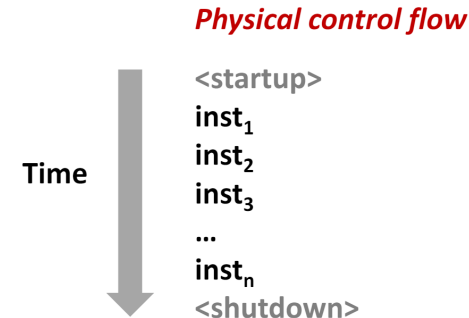
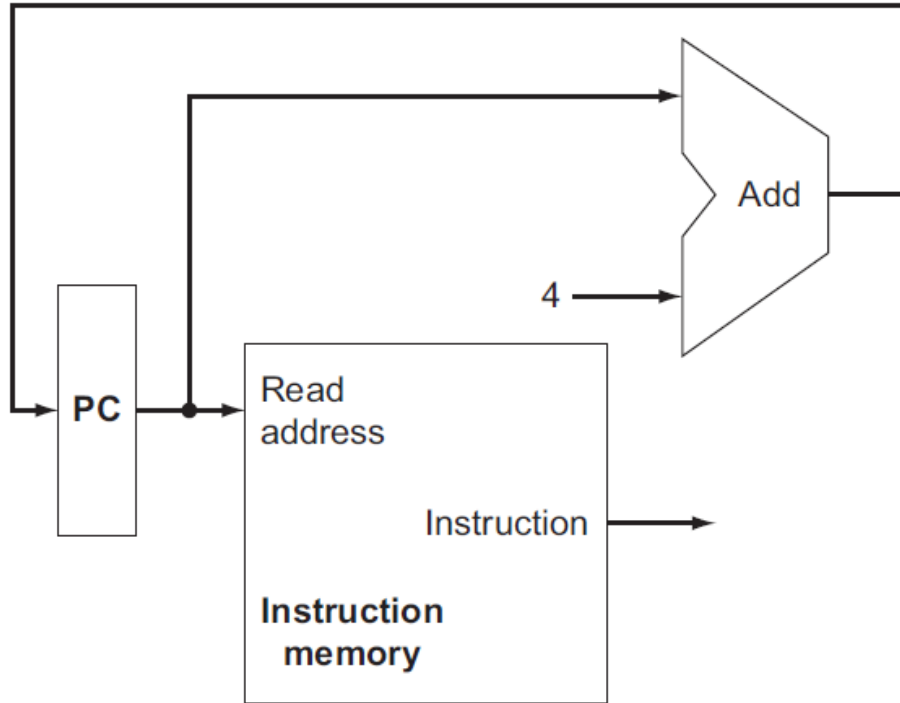


Let's focus on the major components required to execute an ARM instruction:

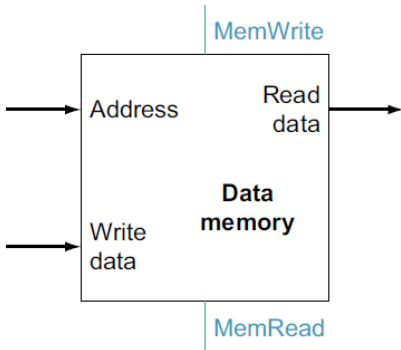
- *memory unit to store the instructions of a program and supply instructions given an address*
- *PC*
- *an adder to increment the PC to the address of the next instruction*
(more on the adders in a few slides!)



Main components for simple instructions

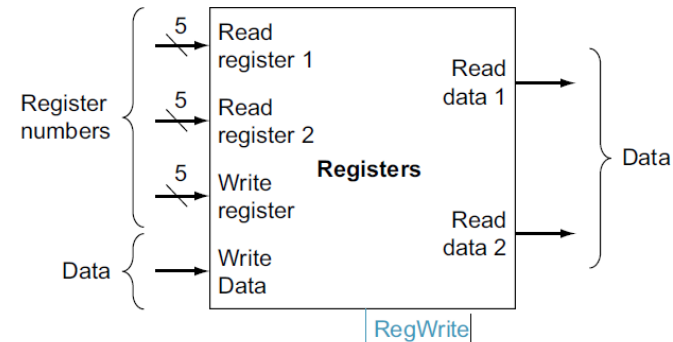


Main components for simple instructions



The **memory unit** is a state element with inputs for the address and the write data, and a single output for the read result. There are separate read and write controls, although only one of these may be asserted on any given clock.

The register file contains all the registers and has two read and one write port



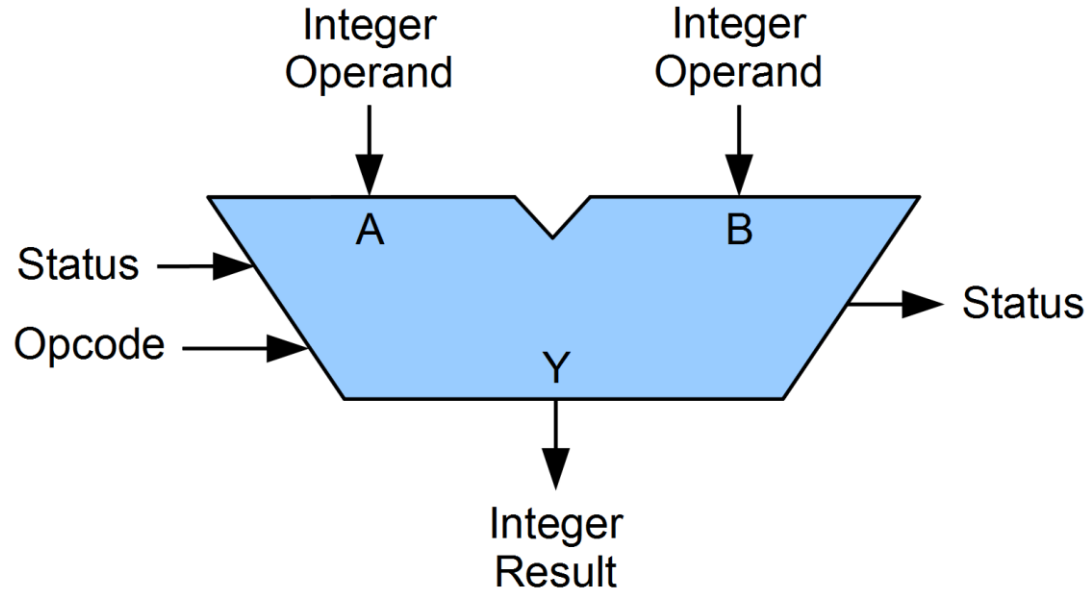
The Arithmetic and Logic Unit (ALU)

ALU

- **The central building blocks of computers**
- Part of the computer that actually performs arithmetic and logical operations on data
- All of the other elements of the computer system are there mainly **to bring data into the ALU** for it to process and then to take the results back out
- Based on the use of simple digital logic devices that can store binary digits and perform simple Boolean logic operations

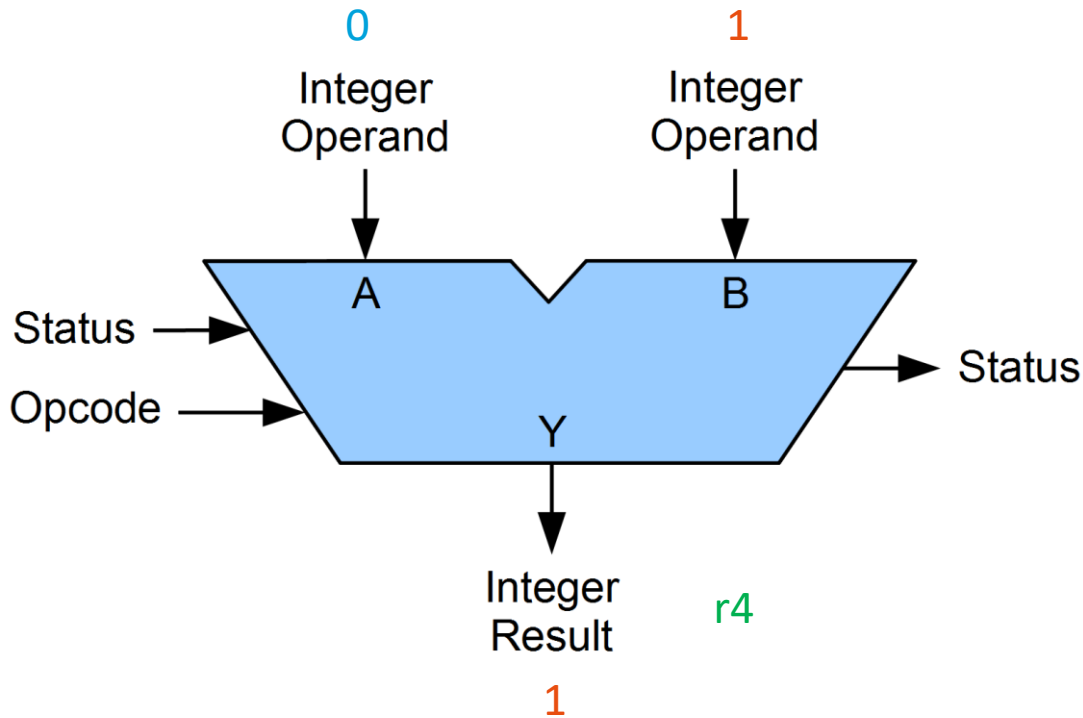


ALU



symbolic representation of an ALU

ALU



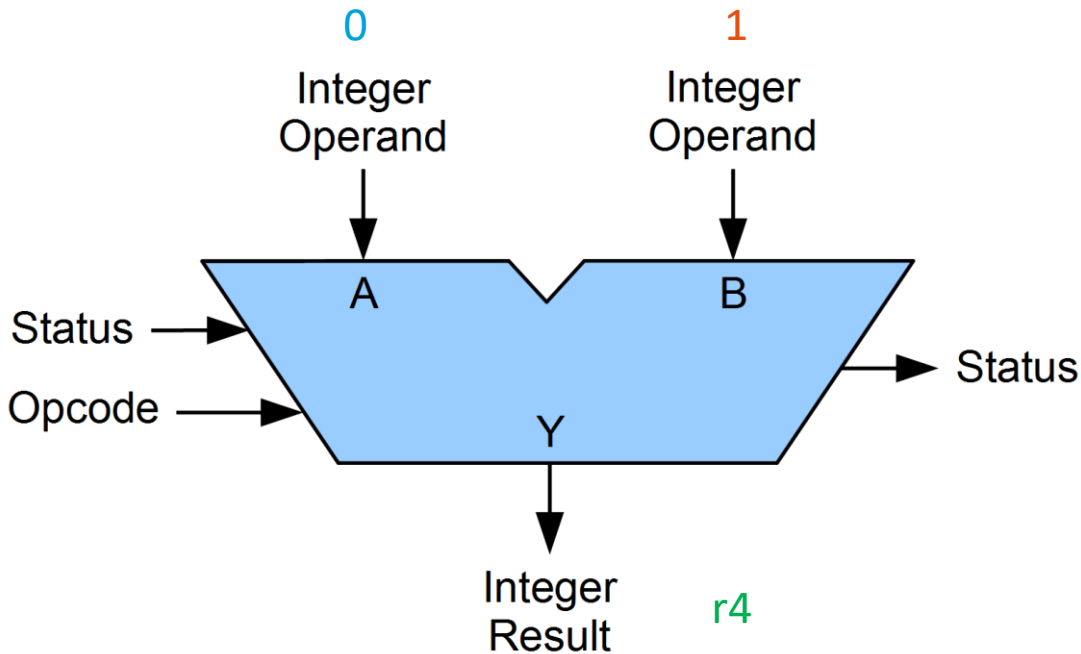
```
register unsigned int var3 = 0;  
var3 = var3 + 1;
```



```
0800028e: 00 24  
08000290: 01 34
```

```
movs    r4, #0  
adds    r4, #1
```

ALU



```
register unsigned int var3 = 0;  
var3 = var3 + 1;
```

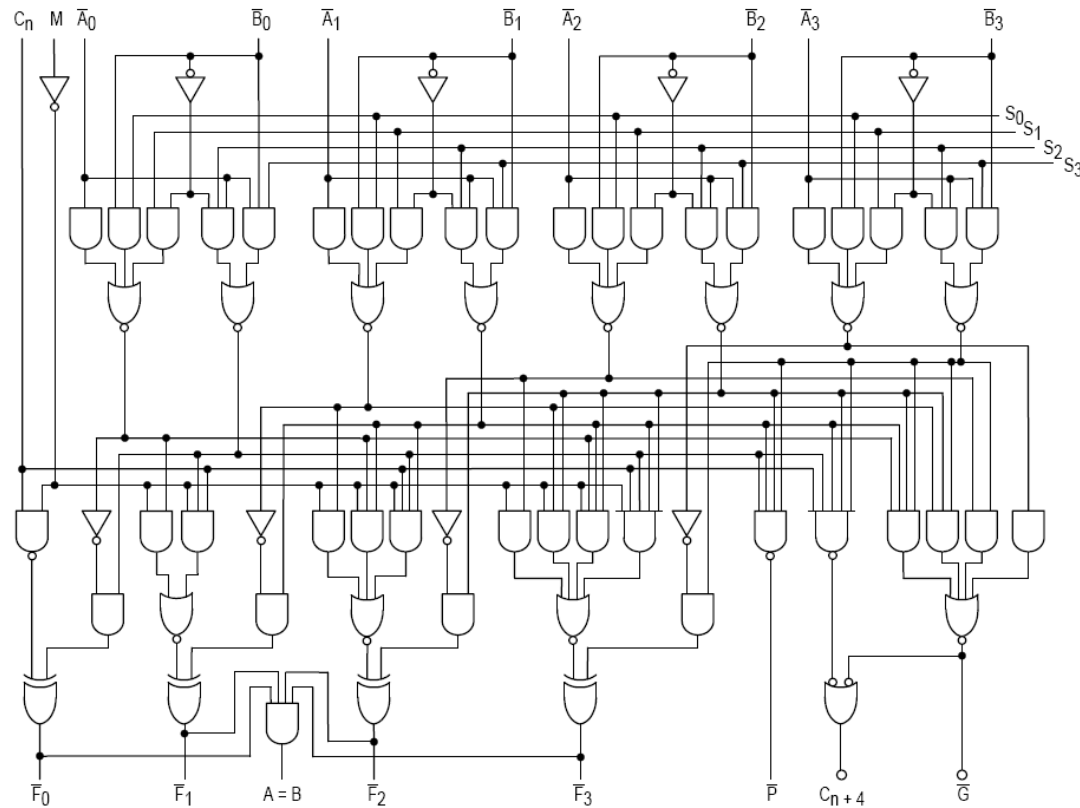


1

0800028e:	00	24
08000290:	01	34

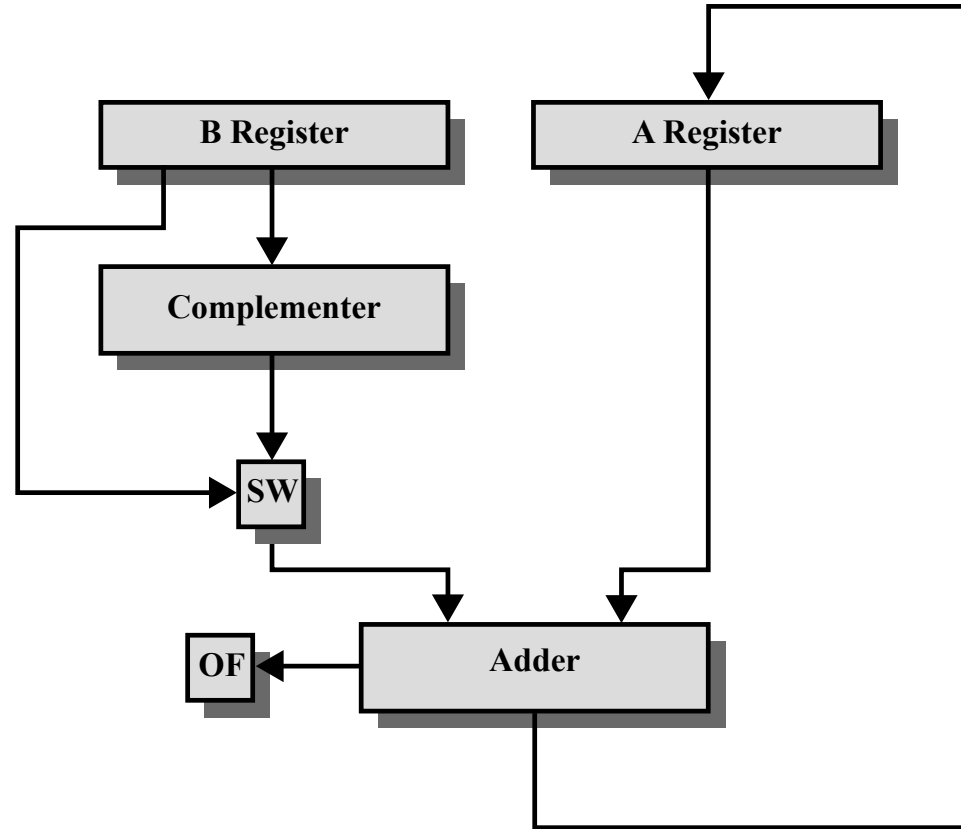
movs	r4,	#0
adds	r4,	#1

ALU



The combinational 4-bit logic circuitry of the 74181 IC. Performs add / subtract / decrement operations with or without carry, as well as AND / NAND, OR / NOR, XOR, and shift

ALU

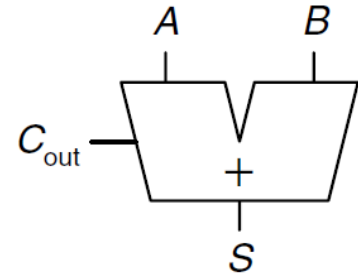


OF = overflow bit

SW = Switch (select addition or subtraction)

ALU – half adder

- Addition is one of the most common operations in digital systems
- We first consider how to add two **1-bit binary numbers**. We then extend to N-bit binary numbers
- We begin by building a 1-bit *half adder* with two inputs, A and B and two outputs, S and C_{out}
- If A and B are both 1, S is 2, which cannot be represented with a single binary digit. Instead, it is indicated with a carry out, C_{out} in the next column



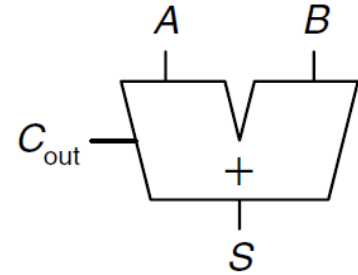
A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

$$C_{out} = AB$$

ALU – half adder

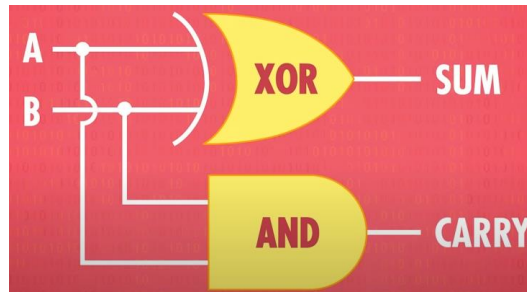
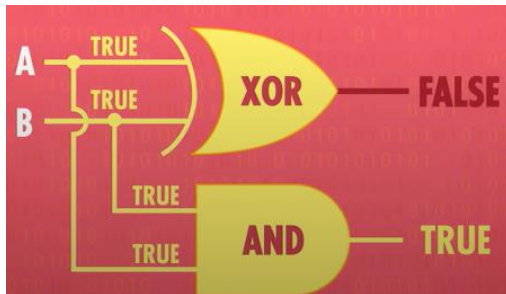
- The half adder can be built from an XOR gate and an AND gate:
 - The first three rows of the truth table correspond to the input/output of an XOR gate
 - The result of the fourth row is *partially* correct for the XOR gate: $1 + 1 = 0$ --- but with an extra carry bit for which we need an **extra bit**



A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$

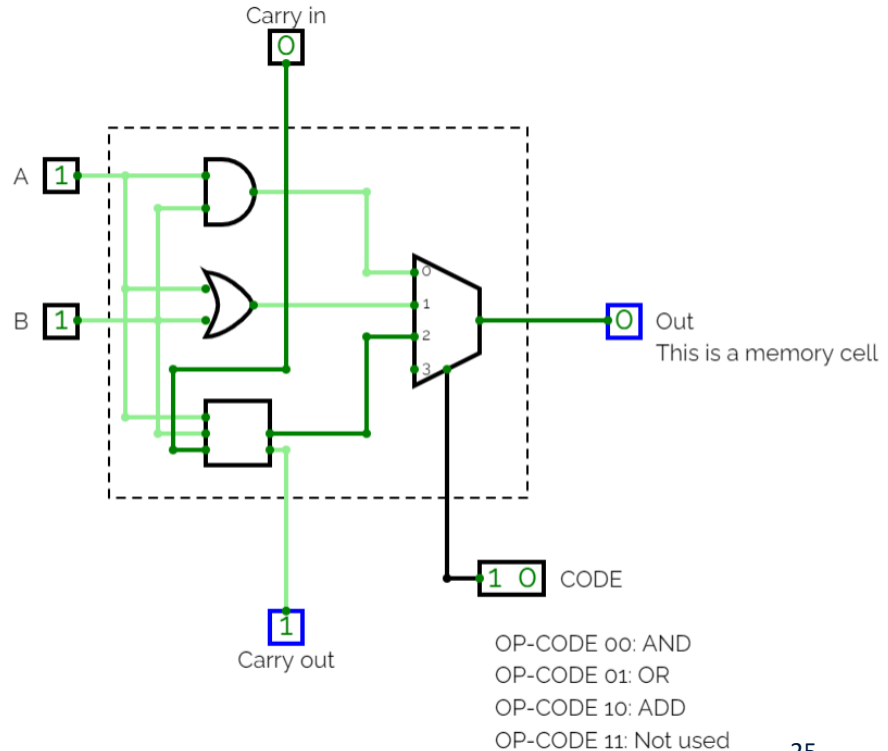
$$C_{out} = AB$$



Experiment with a half adder!

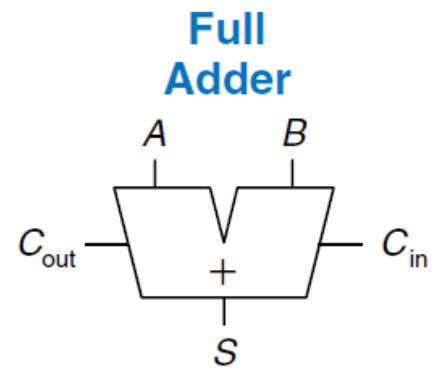
<https://circuitverse.org/simulator/51702>

1-bit with adder



ALU – full adder

- In a multi-bit adder, C_{out} is added or carried to the next most significant bit
- However, the half adder lacks a C_{in} input to accept C_{out} of the previous column. The *full adder* solves this problem



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

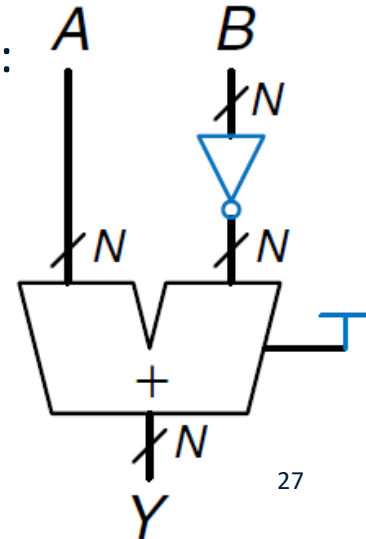
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$



ALU – subtraction

- Adders can add positive and negative numbers using *two's complement* number representation
- Subtraction is almost as easy: flip the sign of the second number, then add. Flipping the sign of a two's complement number is done by inverting the bits and adding 1
- To compute $Y = A - B$, first create the two's complement of B: invert the bits of B to obtain $\bar{B}$ and add 1 (*more on this later!*)
- Add this to A to get $Y = A + \bar{B} + 1$



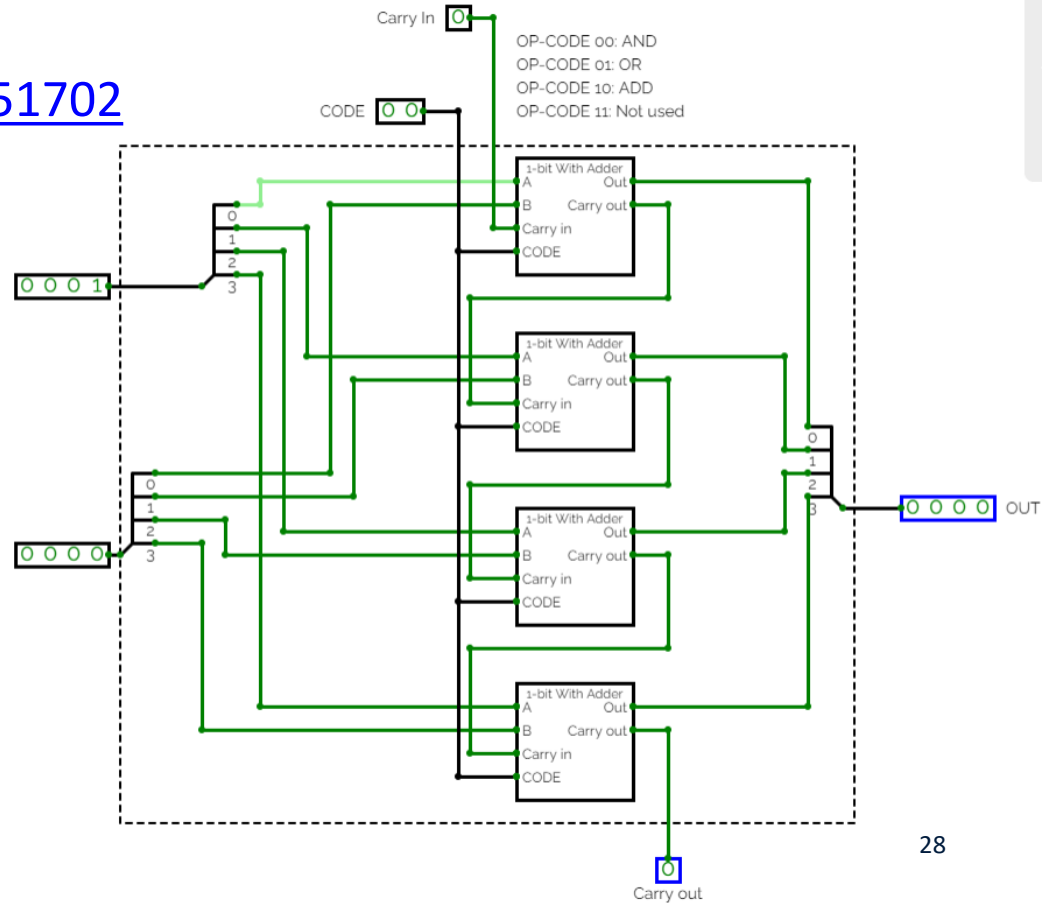
08000292: 01 3c

subs r4, #1

Experiment with a 4-bit ALU!

<https://circuitverse.org/simulator/51702>

4-bit ALU



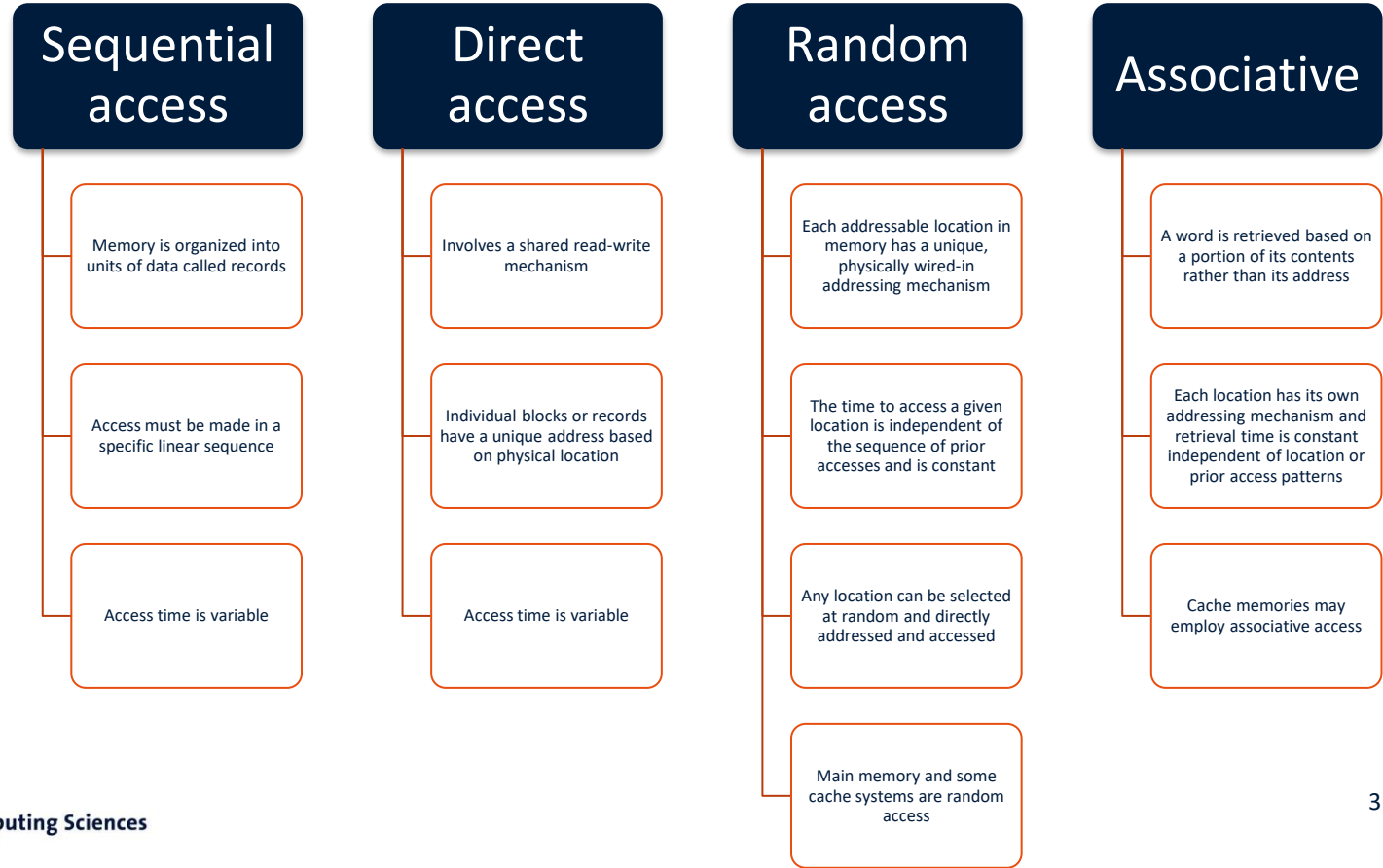
Memory: types and hierarchies

Memory

- From the earliest days of computing, programmers wanted **unlimited** amounts of **fast and cost-effective** memory – these needs are often contradictory!
- There are three primary technologies used today for memories:
 - Main memory is implemented from DRAM (dynamic random-access memory), while levels closer to the processor (caches) use SRAM (static random-access memory)

Memory technology	Typical access time	\$ per GiB in 2020
SRAM semiconductor memory	0.5–2.5 ns	\$500–\$1000
DRAM semiconductor memory	50–70 ns	\$3–\$6
Flash semiconductor memory	5,000–50,000 ns	\$0.06–\$0.12

Memory - access methods

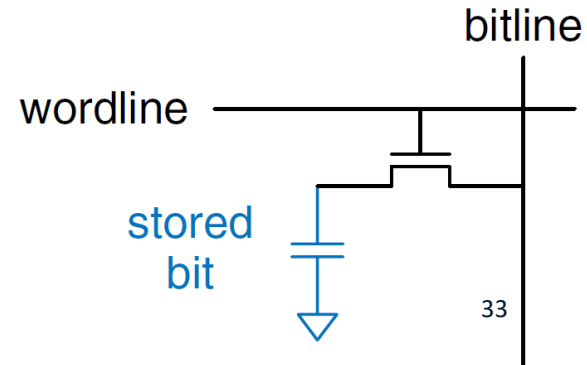


Memory - types

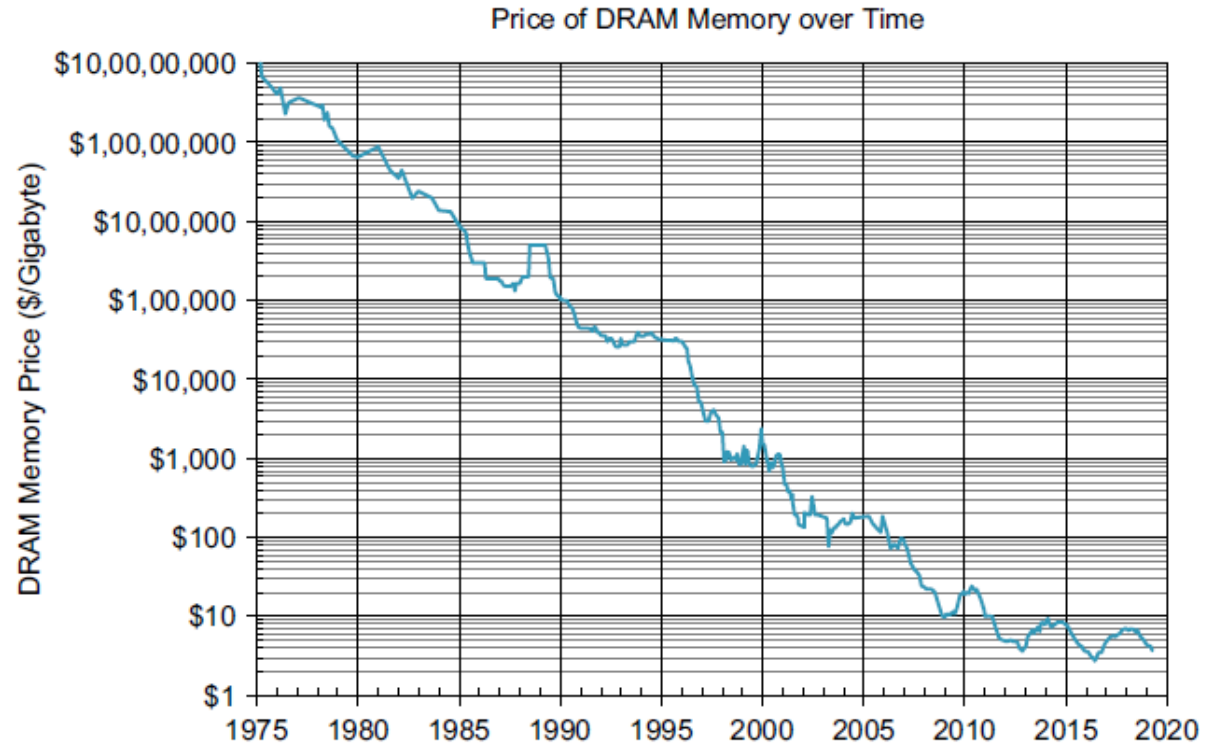
Memory Type	Category	Erasure	Write Mechanism	Volatility
Random-access memory (RAM)	Read-write memory	Electrically, byte-level	Electrically	Volatile
Read-only memory (ROM)	Read-mostly memory	Not possible	Masks Electrically	Nonvolatile
Programmable ROM (PROM)				
Erasable PROM (EPROM)		UV light, chip-level		
Electrically Erasable PROM (EEPROM)		Electrically, byte-level		
Flash memory		Electrically, block-level		

Memory - DRAM

- DRAM stores a bit as the presence or absence of charge on a capacitor
- The transistor in the image behaves as a switch that either connects or disconnects the capacitor from the bitline. When the wordline is asserted, the transistor turns ON and the stored bit value transfers to or from the bitline
- DRAMs use only **one** transistor per bit of storage - much **denser** and **cheaper** per bit than SRAM
- Reading destroys the bit value stored on the capacitor, so the data word must be rewritten after each read
- Even when DRAM is not read, the contents must be refreshed every few milliseconds, because the charge on the capacitor gradually **leaks away**. That is why it is called dynamic, in contrast to the static storage in SRAM

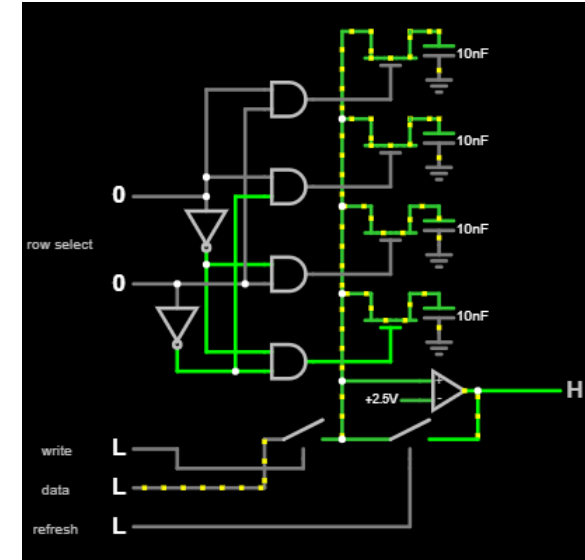


Memory - DRAM



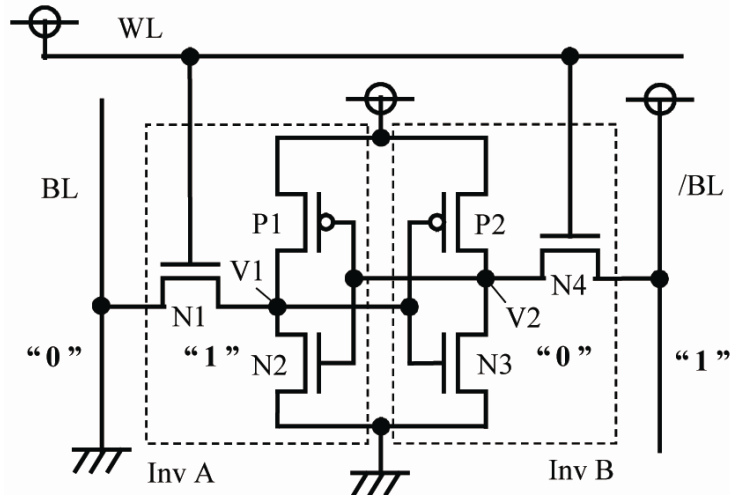
Experiment with a DRAM!

- <https://www.falstad.com/circuit/e-dram.html>
- Very simple model of dynamic RAM, with a capacity of four bits. There is only one column, with four rows
- Select which bit you want with the two inputs labeled "row select". The output is on the right. To write a bit, specify the bit you want to write with the "data" input and then click the "write" input. This will charge (or discharge) the appropriate capacitor
- The capacitors will slowly drain over time, so each row must be refreshed periodically. To do this, select the row and click the "refresh" input.
- Try to right click one of the caps and press "View in New Scope"



Memory - SRAM

- SRAMs are simply integrated circuits that are memory arrays with (usually) a single access port that can provide either a read or a write
- SRAM is inherently faster than DRAM
- SRAMs **don't need to refresh** and typically use **six to eight transistors** per bit
- SRAM needs only minimal power to retain the charge in standby mode

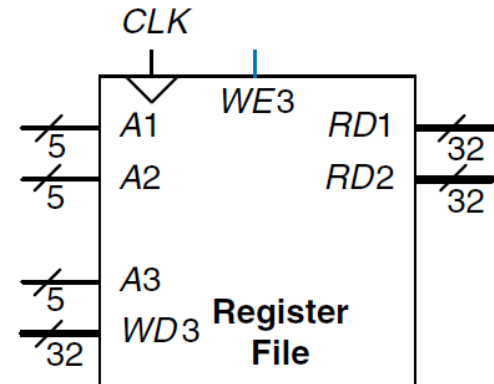


Memory - Flash

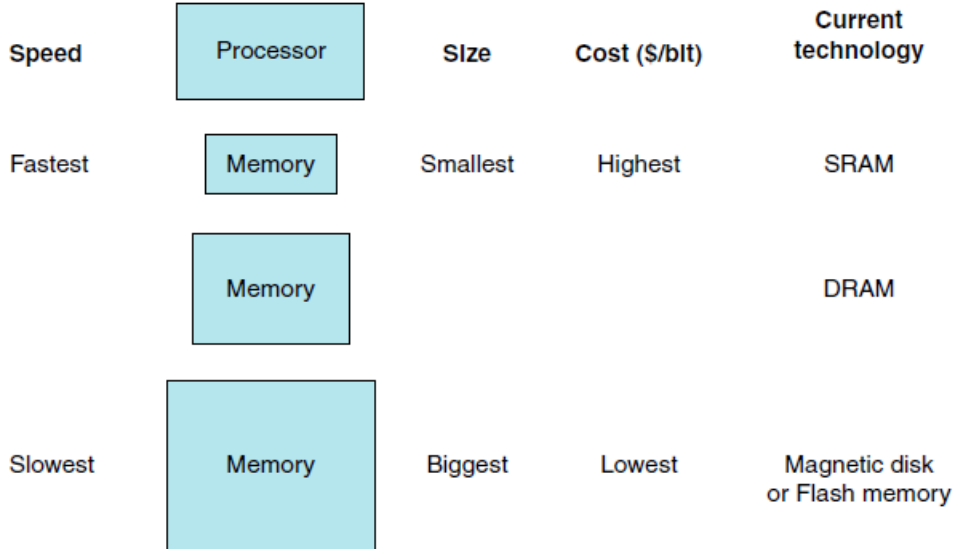
- Flash memory is a type of electrically erasable programmable read-only memory (EEPROM)
- It is based on floating-gate transistors
- It has **much slower speed** compared to DRAM and SRAM and typically used for long-term storage (e.g., SSDs, USB drives, memory cards)
- Unlike DRAM, but like other EEPROM technologies, writes can wear out flash memory bits - **limited write/erase cycles**
- To cope with such limits, most flash products include a controller to spread the writes by remapping blocks that have been written many times to less trodden blocks. This technique is called *wear leveling*

Memory - Registers

- Digital systems often use a number of registers to store temporary variables. This group of registers, or **register file**, is usually built as a small **SRAM** array
- Registers are primitives used in hardware design that are also visible to the programmer when the computer is completed, so you can think of registers as the bricks of computer construction
- The size of a register in the ARMv8M architecture is 32 bits; groups of 32 bits are given the name word while groups of 64 bits are called a doubleword



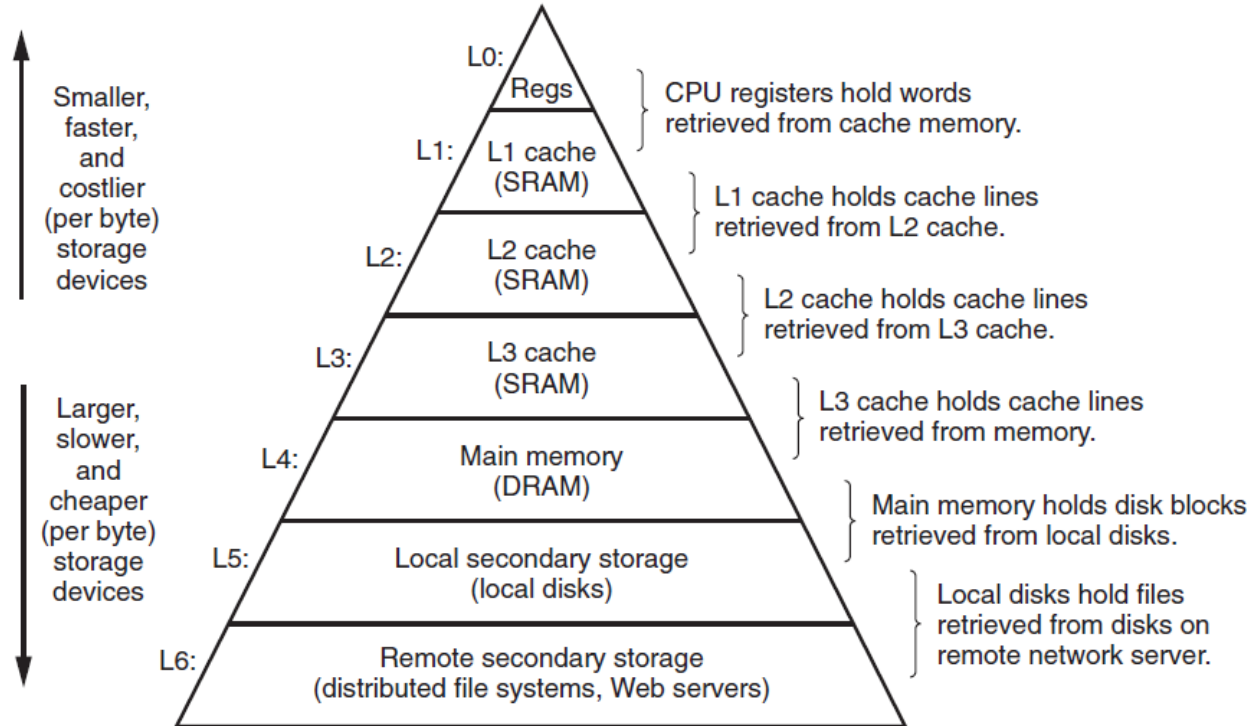
Memory - Hierarchies



Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

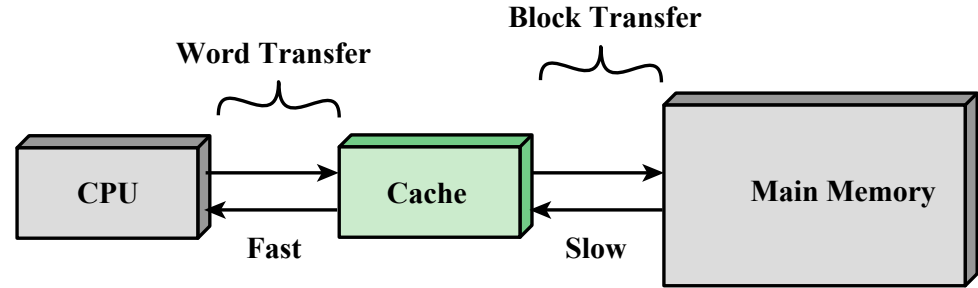
By implementing the memory system as a hierarchy, the user has the illusion of a memory that is as large as the largest level of the hierarchy, but can be accessed as if it were all built from the fastest memory

Memory - Hierarchies

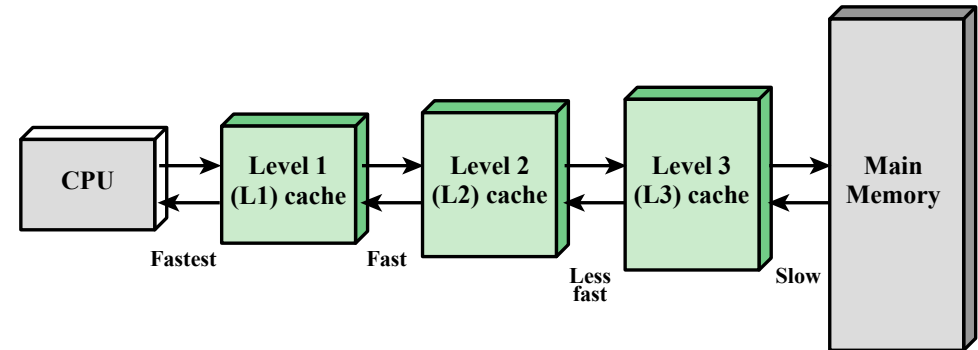


Memory - Hierarchies

By implementing the memory system as a hierarchy, the user has the illusion of a memory that is as large as the largest level of the hierarchy, but can be accessed as if it were all built from the fastest memory



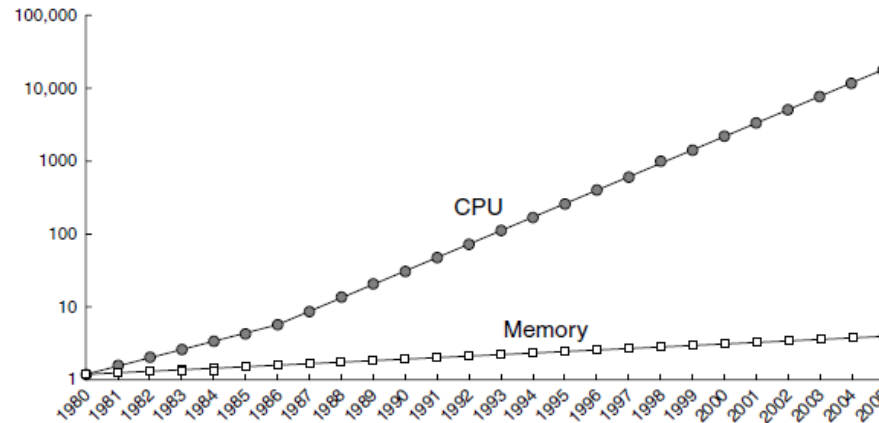
(a) Single cache



(b) Three-level cache organization

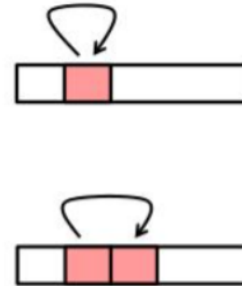
Memory - Caches

- Computer memory is generally built from DRAM chips – they are cheap!
- However, DRAM access time is one to two orders of magnitude longer than the processor cycle time :(
- Computers store the most commonly used instructions and data in a faster but smaller SRAM memory, called a cache



Cache – Principle of locality

- Caches operate on the principle of locality: “programs access a relatively small portion of their address space at any instant of time”
 - **Temporal locality (locality in time):** *if an item is referenced, it will tend to be referenced again soon*
 - **Spatial locality (locality in space):** *if an item is referenced, items whose addresses are close by will tend to be referenced soon*



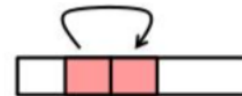
Cache – Principle of locality

```
int sum = 0;  
int x[1000];
```

```
for (int i = 0; i < 1000; i++) {  
    sum += i;  
    x[i] = 0;  
}
```

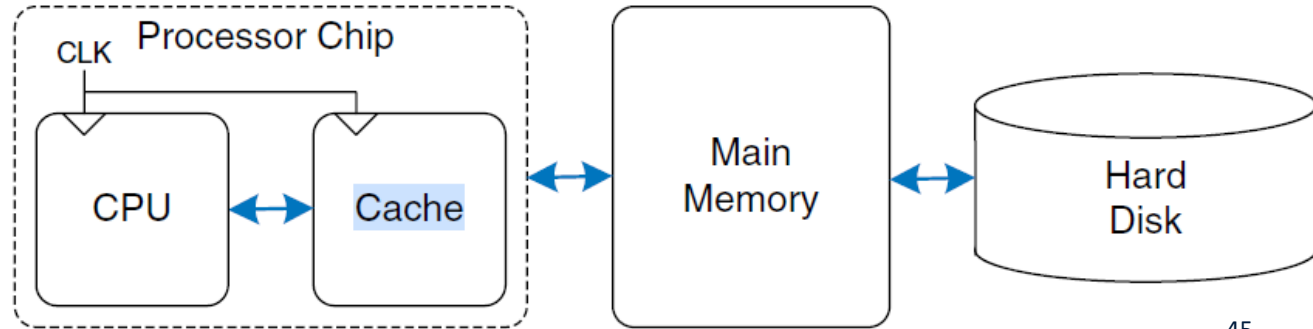
Which memory accesses demonstrate temporal locality?

Which memory accesses demonstrate spatial locality?

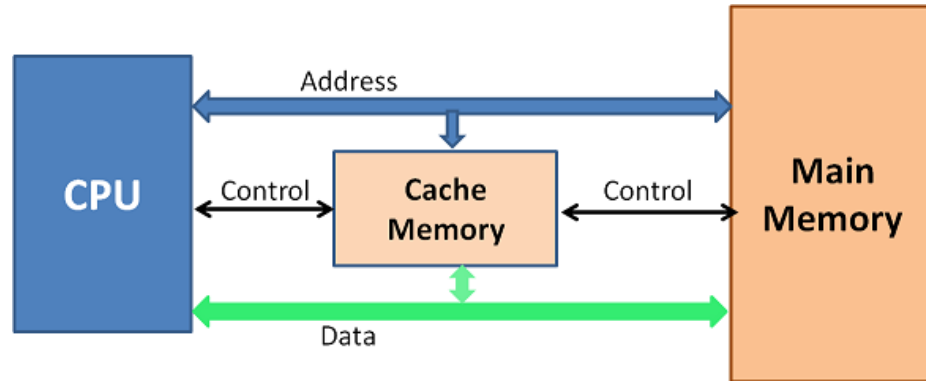


Memory - Caches

- Caches can store both instructions and data, **but** we will refer to their contents generically as “**data**”
- If the processor requests data that is available in the cache, it is returned quickly. This is called a **cache hit**. Otherwise, the processor retrieves the data from main memory (DRAM). This is called a **cache miss**
- If the cache hits most of the time, then the processor seldom has to wait for the slow main memory, and the average access time is low



Memory - Caches



Cache – Mapping function

- There are fewer cache lines than main memory blocks, so an algorithm is needed for mapping main memory blocks into cache lines
- Three techniques are commonly used:

Direct

- The simplest technique
- Maps each block of main memory into only one possible cache line

Associative

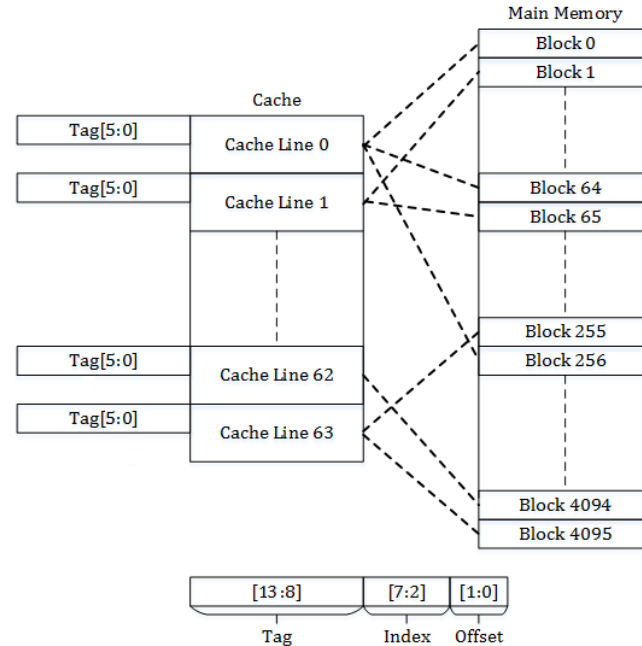
- Permits each main memory block to be loaded into any line of the cache
- The cache control logic interprets a memory address simply as a Tag and a Word field
- To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's Tag for a match

Set Associative

- A compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages

Direct mapped cache

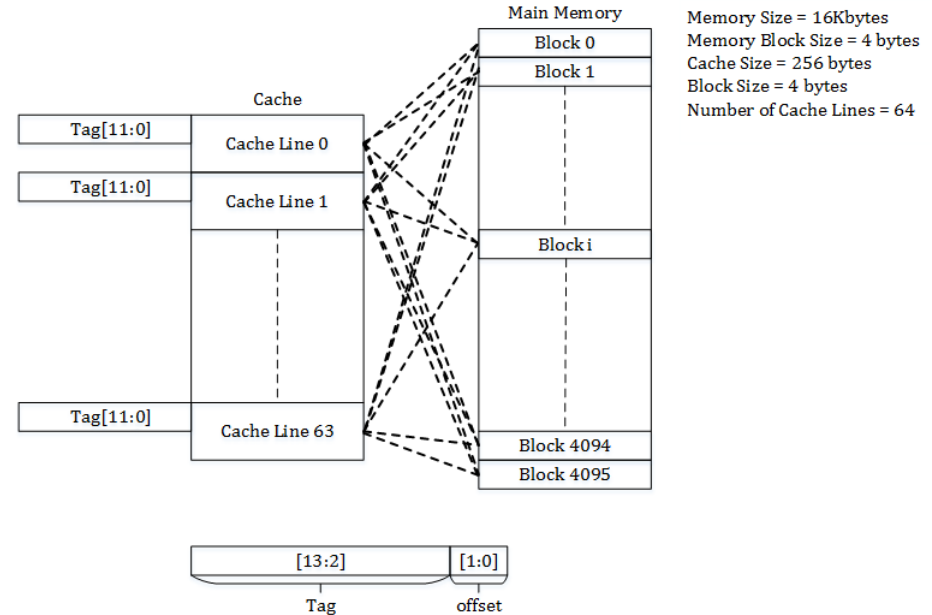
- Mapping of main memory to a direct mapped cache
- The cache is organized into multiple sets with a single cache line per set
- **Simple placement policy and replacement policy**
- Lower cache hit rate, as there is only one cache line available in a set



Memory Size = 16Kbytes
Memory Block Size = 4 bytes
Cache Size = 256 bytes
Block Size = 4 bytes
Associativity = 1
Number of Sets = 64

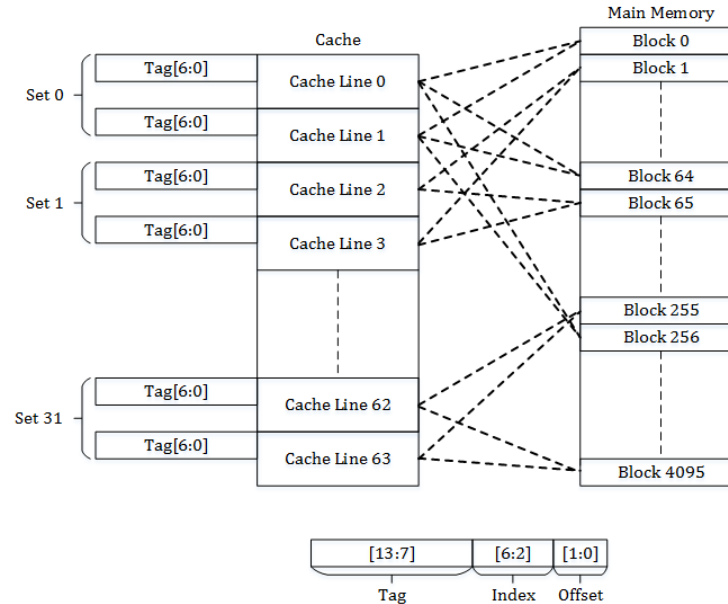
Fully associative cache

- Cache organized into a single cache set with multiple cache lines. A memory block can occupy any cache line
- **Flexibility** of placing a memory block in any cache line and hence full utilization of the cache. Better cache hit rate
- **Power hungry** as the comparison circuitry has to run over the entire cache to locate a block
- The **most expensive** of all methods, due to the high cost of associative-comparison hardware



Set associative cache

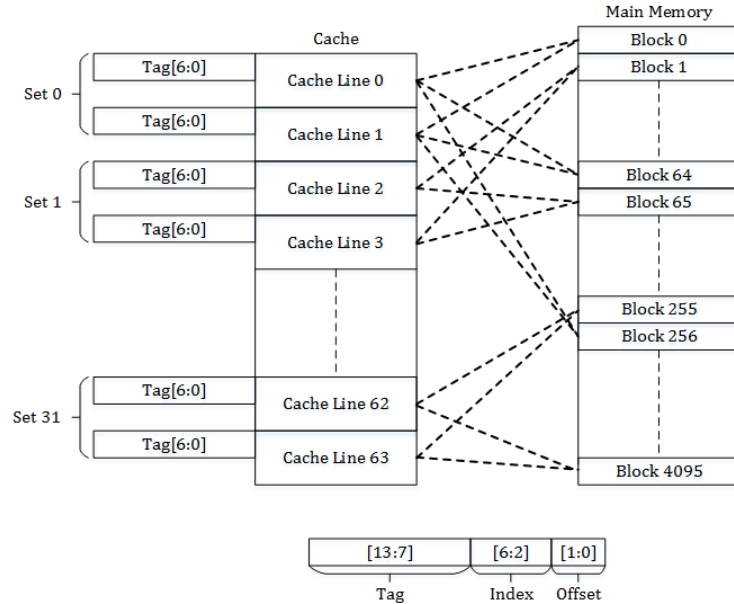
- Cache divided in 'n' sets (32 in this example), each set with 'm' cache lines (2 in this example).
- A memory block is first mapped onto a set and then placed into any cache line of the set
- Trade-off between direct-mapped and fully associative cache



Memory Size = 16Kbytes
Memory Block Size = 4 bytes
Cache Size = 256 bytes
Block Size = 4 bytes
Associativity = 2
Number of Sets = 32

Set associative cache

- Example: 2-Way Set Associative Cache
- If a cache has 64 lines and is 2-way set associative, it forms 32 sets, with each set containing 2 slots (ways).
- A memory block can map to only one of the 32 sets but can be stored in either of the two ways in that set.



Memory Size = 16Kbytes
Memory Block Size = 4 bytes
Cache Size = 256 bytes
Block Size = 4 bytes
Associativity = 2
Number of Sets = 32

Cache exercise

You have a cache with 8 lines, and each line can store one word. The main memory has 64 words, numbered 0 to 63. Determine the cache line for each memory address in:

- Direct-Mapped Cache
- Fully Associative Cache
- 2-Way Set-Associative Cache

Cache exercise

You have a cache with 8 lines, and each line can store one word. The main memory has 64 words, numbered 0 to 63.

- For direct-mapped cache, use the formula:

$$\text{Cache line} = \text{Memory Address} \% \text{ Number of cache lines}$$

- For fully associative cache, any memory address can be mapped to any cache block. No need to calculate
- For a 2-way set-associative cache: Divide the cache into 4 sets (each set has 2 blocks). Use the formula to determine the set:

$$\text{Set} = \text{Memory Address} \% \text{ Number of Sets}$$

Cache exercise

You have a cache with 8 lines, and each line can store one word. The main memory has 64 words, numbered 0 to 63.

Example:

- For memory Address 10
- Direct-mapped: $10 \% 8 = 2$ (Cache line 2)
- Fully associative: Can go to any block
- 2-Way Set-Associative: $10 \% 4 = 2$ (Set 2, pick one of the 2 blocks in the set)

Replacement algorithms

- Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced
- For direct mapping there is only one possible line for any particular block and **no choice is possible**
- For the associative and set-associative techniques a **replacement algorithm** is needed
- To achieve high speed, an algorithm must be implemented in hardware

Replacement algorithms

- **First-in-first-out (FIFO)**
 - Replace that block in the set that has been in the cache **longest**
 - Easily implemented as a round-robin or circular buffer technique
- **Least recently used (LRU)**
 - Most effective
 - Replace the block in the set that has been in the cache **longest** with no reference to it
 - Because of its simplicity of implementation, LRU is the most popular replacement algorithm
- **Least frequently used (LFU)**
 - Replace that block in the set that has experienced the fewest references
 - Could be implemented by associating a counter with each line

Replacement algorithms exercise

Given the memory access sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Compare the **FIFO** and **LRU** replacement policies for a cache with 4 lines:

- Simulate both policies
- Count the total hits and misses for each policy

Replacement algorithms exercise

Given the memory access sequence: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Compare the **FIFO** and **LRU** replacement policies for a cache with 4 blocks:

- Simulate both policies
- Count the total hits and misses for each policy

FIFO			
Step	Memory access	Cache content	Hit/Miss
1	7	7	Miss
2	0	7, 0	Miss
3	1	7, 0, 1	Miss
4	2	7, 0, 1, 2	Miss
5	0	7, 0, 1, 2	Hit
6	3	0, 1, 2, 3	Miss
7	0	0, 1, 2, 3	Hit
8	4	1, 2, 3, 4	Miss
9	2	1, 2, 3, 4	Hit
10	3	1, 2, 3, 4	Hit
11	0	2, 3, 4, 0	Miss
12	3	2, 3, 4, 0	Hit
13	2	2, 3, 4, 0	Hit

LRU			
Step	Memory access	Cache content	Hit/Miss
1	7	7	Miss
2	0	7, 0	Miss
3	1	7, 0, 1	Miss
4	2	7, 0, 1, 2	Miss
5	0	7, 1, 2, 0	Hit
6	3	1, 2, 0, 3	Miss
7	0	1, 2, 3, 0	Hit
8	4	2, 3, 0, 4	Miss
9	2	3, 0, 4, 2	Hit
10	3	0, 4, 2, 3	Hit
11	0	4, 2, 3, 0	Hit
12	3	4, 2, 0, 3	Hit
13	2	4, 0, 3, 2	Hit

Takeaways

- The Von Neumann and Harvard architectures
- The CPU: control, ALU and registers
- ALU: half and full adders, subtraction
- Memory types and access methods
- SRAM, DRAM and Flash
- Memory model – storing data and instructions
- Memory caches